ThreatForge: An ATT&CK-Aligned Framework for Adversary Emulation and Closed-Loop Detection Validation in Smart-Home IoT

Author,

GIRIDHARAN VEDA PARTHIBAN
Information Security Researcher
Department of Information Technology
Somaiya Vidyavihar University, Mumbai, Maharashtra – India.

ORCID ID: 0009-0000-9218-8028 | Scopus ID: 59652720900 | Researcher ID: QUU-4601-2026

Abstract

Consumer IoT devices like smart cameras, locks, bulbs, and hubs have expanded the attack surface of residential and enterprise networks. Existing adversary-emulation platforms, including MITRE CALDERA, remain focused on enterprise IT systems and offer limited support for IoT-specific protocols, so organisations deploying IoT devices alongside SIEM platforms lack a way to confirm if realistic IoT attacks will be detected before production. This paper presents ThreatForge, a protocol-aware adversary-emulation framework that closes the loop between ATT&CK-aligned attack execution, IoT telemetry generation, SIEM-based detection, and ground-truth validation. Unlike a simple co-deployment of existing emulation, testbed, and SIEM tools, ThreatForge provides the integration layer needed to connect protocol-aware attack execution with IoT telemetry and SIEM detection in a closed-loop workflow. ThreatForge extends MITRE CALDERA with a custom agent supporting native HTTP, MQTT, and SSH executors, integrates this agent with a Node-RED smart-home digital twin, and a Wazuh SIEM layer configured with custom, MITRE-mapped detection rules. A five-stage, ATT&CK-aligned adversary profile spanning reconnaissance, collection, impact, and lateral movement was executed in 25 trials. Across these trials, the framework achieved a mean detection coverage of 94.4% ± 9.2%, a precision of 89.8% ± 10.4%, an F1-score of 91.6% ± 7.5%, a mean detection latency of (59.0 ± 7.0) s, and a low false-positive count of 0.60 ± 0.65 alerts per run. This implies trustworthy IoT security evaluation depends on validating the complete monitoring pipeline rather than attack execution alone. ThreatForge has been released publicly to support reproducible IoT detection engineering and security research.

Keywords: IoT security, adversary emulation, MITRE ATT&CK, SIEM detection, closed loop validation.

1. Introduction

1.1. Context and Importance

IoT has deeply embedded smart devices within residential and enterprise environments, creating a device layer that broadens the attack surface available to adversaries. These devices typically have limited computing resources and are frequently left running outdated or absent security patches. Consumer-grade smart-home ecosystems illustrate this exposure concretely: they expose device-inventory endpoints, weakly authenticated control interfaces, and messaging channels that were never designed with an adversarial model in mind [1], [2], making IoT ecosystems soft, attractive targets for adversaries [3].

1.2. Problem Statement

Despite this risk, organisations that deploy IoT devices alongside SIEM platforms lack a reliable method for confirming, before production, that an attack against a specific IoT device or protocol will be detected. Manual red-team exercises against IoT hardware remain expensive, labour-intensive, and difficult to reproduce at scale [4]. Detection logic designed purely on paper carries a risk: rules crafted against assumed attacker behaviour and never exercised against a live attack chain can appear sound on review but fail in operation due to log-format mismatches, incorrect field mappings, or rule-ordering errors. What is missing is a mechanism that closes the loop between an executed attack and the detection outcome it produces.

1.3. Existing Work and Limitations

Security operations teams have relied on log correlation and rule-based alerting platforms such as Wazuh as their primary line of detection [4], while adversary-emulation platforms such as MITRE CALDERA allow defenders to execute MITRE ATT&CK-mapped techniques in controlled environments to validate detection controls [5]. These tools have proven effective in enterprise IT contexts, but their application has remained concentrated on Windows, Linux endpoints, Active Directory, and cloud workloads. Existing CALDERA deployments, in particular, lack the native executor commands needed to interact with IoT-specific protocols and interfaces [6], leaving a portion of the modern attack surface outside the reach of established emulation-and-validation workflows.

1.4. Research Gap

These limitations point to a clear unaddressed gap: the absence of an IoT-specific adversary-emulation capability integrated with a closed-loop detection-validation process. Existing frameworks offer neither the protocol-aware execution needed to realistically emulate adversary attacks against IoT device interfaces and messaging channels nor a way to link an executed attack chain to the telemetry and alerts it should generate. Consequently, no repeatable method exists for evaluating whether a defensive stack will detect an IoT attack chain before it is encountered in production.

This gap motivates three research questions that guide the present work: (RQ1) whether ATT&CK-aligned behaviour can be realistically reproduced in smart-home IoT environments via native protocol-aware execution; (RQ2) whether correlating executed techniques with endpoint telemetry and SIEM alerts enables quantitative validation of detection coverage, precision, and false-positive behaviour; and (RQ3) what detection latency is achievable across different telemetry pathways in a closed-loop pipeline. These questions helped in designing and experimenting with ThreatForge.

1.5. Proposed Solution

To address this gap, this work introduces **ThreatForge**, an IoT-focused framework designed to support both adversary emulation and detection validation. Rather than simply combining CALDERA, Wazuh, and Node-RED. ThreatForge introduces an integration layer required to execute IoT-specific attacks through custom protocol executors and correlates the resulting adversary activity with testbed telemetry and SIEM alerts. This closed-loop design provides a link between each executed attack technique and its detection outcome, enabling the effectiveness in IoT security monitoring that is evaluated across repeated attack scenarios.

1.6. Contributions

Existing adversary-emulation frameworks provide limited support for reproducing IoT-specific attacks across the communication layers. To address this gap, this study makes the following contributions:

1. **An IoT-oriented adversary-emulation agent** is developed to enable attack execution against smart-home devices and their network interfaces, enabling adversary operations that are not directly supported by conventional enterprise-focused agents.
2. **HTTP, MQTT, and SSH protocol executors** are implemented to support attacks through IoT communication channels. These executors provide direct interaction with device interfaces and communication services, extending adversary emulation to protocol-level interactions.
3. **A set of IoT-specific abilities and an adversary profile** is developed to execute a multi-stage attack chain against smart-home devices covering reconnaissance, collection, impact, and lateral movement.
4. **Custom detection rules for IoT attacks** are designed and integrated with Wazuh SIEM to map executed adversary techniques to the telemetry they generate and the corresponding security alerts, providing a direct link between attack activity and detection outcomes.
5. **An evaluation methodology** is established to measure detection coverage, precision, and detection latency across repeated trials by correlating adversary execution, IoT telemetry, and SIEM alerts.

ThreatForge is an open-source framework to combine ATT&CK-aligned adversary emulation with detection validation specifically for consumer smart-home IoT environments. The complete framework has been released publicly to support IoT security research.

2. Related Work and Literature Review

2.1. Adversary Emulation in IoT and Smart-Home Environments

Cybersecurity practice has shifted from incident response to validation through adversary emulation. Rather than relying on post-incident analysis or isolated penetration testing, this approach recreates the tactics, techniques, and procedures (TTPs) used by real attackers, enabling direct testing of defensiveness [7]. Ajmal et al. demonstrate that replaying attacker behaviour against live systems supports proactive threat hunting [7]. The MITRE ATT&CK framework underpins much of this shift, offering a structured knowledge base that models adversary behaviour as observable techniques [8]. MITRE CALDERA operationalises this model by translating ATT&CK techniques into executable, repeatable actions [5]. Other work frames automated emulation as a planning-and-execution problem shaped by environmental dynamics and not because of static scripting [2].

Orbinato et al.'s Laccolith, for instance, uses hypervisor-assisted techniques to evade endpoint defences during simulated attacks [9], and comparative platform analyses consistently rank CALDERA highly for its ATT&CK integration and modularity [6]. SpecRep addresses part of the automation burden by generating attack workflows directly from high-level objectives across infrastructures [10]. Despite this progress, these platforms remain enterprise-oriented, offering limited native support for protocol-aware IoT experimentation [6]. Most studies judge emulation success from the attacker's command-and-control perspective, paying little attention to whether the actions are observable in defensive telemetry.

2.2. IoT Smart-Home Threat Landscape and Network-Level Vulnerabilities

Smart-home environments function as devices that communicate over lightweight protocols, extending the exploitable surface well beyond conventional endpoints [1]. Network traffic analysis has proven effective for fingerprinting IoT devices and characterising their behaviour [11]. Protocol-level studies converge on a common theme: vulnerabilities frequently originate in messaging infrastructure rather than device firmware. Wang et al. identify this pattern in MQTT-based publish-subscribe systems [12], and Jia et al. reach a parallel conclusion from the cloud side: design-level weaknesses in general-purpose IoT messaging protocols persist regardless of vendor-specific firmware [3].

Platforms such as IoT Inspector illustrate the value of large-scale measurement in this effort [13]. This work remains largely data-centric; as a result, the link between executed ATT&CK techniques, the telemetry they generate, and the detections they trigger is rarely examined in reproducible smart-home settings; protocol analysis and attack-execution validation remain largely separate lines of inquiry.

2.3. Detection Engineering and SIEM-Based Monitoring

Detection engineering has become central to Security Operations Centres. Rather than relying on signatures, it translates adversary behaviour into measurable detection logic through ATT&CK-aligned analytics and telemetry correlation. Among open-source SIEM platforms, Wazuh has attracted particular attention for its ATT&CK mapping, centralised log collection, and custom rule support [4]. Integrations pairing Wazuh with CALDERA show that ATT&CK-aligned emulation can validate both built-in and custom detection logic, supporting purple-team workflows [4].

Reproducibility remains a persistent weakness in this space, however. Industrial control system testbeds have combined ATT&CK-based execution with SIEM monitoring to measure detection coverage under controlled conditions [14], [15]. Repeated trials, statistical variability, and cross-validation against telemetry sources are uncommon. Without custom detection mechanisms and without repeatable measurement of coverage, precision, and time to detection, claims of detection efficacy are difficult to trust, let alone reproduce.

2.4. Cyber Ranges, Digital Twins, and Reproducible Security Evaluation

Cyber ranges enable evaluation of attack scenarios without risking production systems [14], [16]. Traditional implementations, however, depend on physical infrastructure, which limits scalability and accessibility [14], [15]. Because smart-home systems handle both IT and operational-technology domains, the attack techniques relevant to this space are drawn from both the Enterprise and ICS ATT&CK matrices [17]. This allows researchers to automatically run realistic, multi-step attacks that move from basic network entries all the way to controlling physical smart devices [18], [19]. Ultimately, this approach moves security testing out of pure theory and into a practical hands-on evaluation for everyday connected homes [7], [20].

Software-defined cyber ranges and digital twins offer a more scalable alternative, reproducing realistic network behaviour while supporting automated emulation [14]. Digital-twin environments show that virtualised IoT systems can support security experimentation without large-scale physical deployment [18]. Hybrid IoT cyber ranges take this further, combining virtual devices, automation frameworks, and realistic protocols to enable repeatable experimentation [15]. Even so, many of these platforms prioritise attack execution over defensive validation; MITRE's own CALDERA-for-OT plugin illustrates the pattern, executing OT-aligned campaigns but providing no defensive-validation

layer [19]. Telemetry correlation across IoT environments therefore remains only partially addressed [6] — a gap between the digital twins and their actual use for closed-loop detection assessment.

2.5. Integrated IoT Security Evaluation: Research Gap and Motivation

Adversary-emulation platforms have matured in enterprise contexts [4], [7], [14]; IoT research has deepened understanding of protocol-level vulnerabilities [12]; SIEM platforms support sophisticated detection logic [4]; and digital twins have improved the scalability of experimental environments [15], [18]. Each of these domains has developed in isolation: emulation platforms remain concentrated on enterprise and OT systems rather than smart-home environments [6]; IoT security research examines detection without integrating ATT&CK-aligned execution [12]; and detection-engineering studies rarely validate attacks through telemetry or statistically rigorous repeated trials, a limitation that persists even in widely adopted tooling such as CALDERA-for-OT [19]. Huang et al.'s ARIoTEDef illustrates the value of sequential attack in multi-step IoT attacks, yet it falls short in closing this loop [20].

The literature provides the components needed to evaluate IoT security but does not offer a way to examine how these components perform during an attack. Adversary emulation can reproduce attacker behaviour, while IoT studies reveal protocol-specific weaknesses and detection systems provide mechanisms for identifying malicious activity. The challenge lies in bringing these capabilities into the same experimental setting and establishing how an emulated technique is reflected in telemetry and detected by the monitoring system. This becomes particularly important when detection performance is assessed across repeated trials and not from a single successful execution. The reviewed work therefore points towards the need for a controlled IoT environment in which adversary behaviour, protocol activity, telemetry, and detection outcomes can be evaluated together, enabling a more reliable assessment of how effectively security monitoring responds to realistic IoT attack scenarios.

Tab. 1 maps each strand of prior work. No reviewed study combines full IoT attack execution, ATT&CK-aligned emulation, SIEM-assisted detection, and a closed-loop reproducible framework [6], [15], [19]. This is exactly the absence in the research gap that ThreatForge is created to close.

Table 1. Feature comparison of closely related work and the proposed framework.

Yes: Supports natively. Partial: Supports with limitations or additional integration. No: Not supported

Research Focus	**Ajmal et al. [7]**	**ARIoTEDef [20]**	**Winkler & Sharma [4]**	**Landauer et al. [6]**	**Integrated Testbed [15]**	**ThreatForge (Proposed)**
ATT&CK-based emulation	Yes	No	Partial	Yes	Partial	Yes
IoT environment	No	Yes	No	No	Partial	Yes
IoT attack execution	No	Partial	No	No	Yes	Yes
SIEM-assisted detection	No	No	Yes	No	Partial	Yes
Performance evaluation	Partial	Yes	Partial	Partial	Partial	Yes
Reproducible framework	No	No	No	Partial	Partial	Yes

3. Research Methodology and System Design

3.1. Design Overview

The proposed framework is organised into six architectural layers that integrate IoT simulation, adversary emulation, attack execution, and security monitoring into a single system. Each layer performs one function and passes its output to the next, so that any attack executed against the IoT environment can be followed from initiation through to its detection outcome.

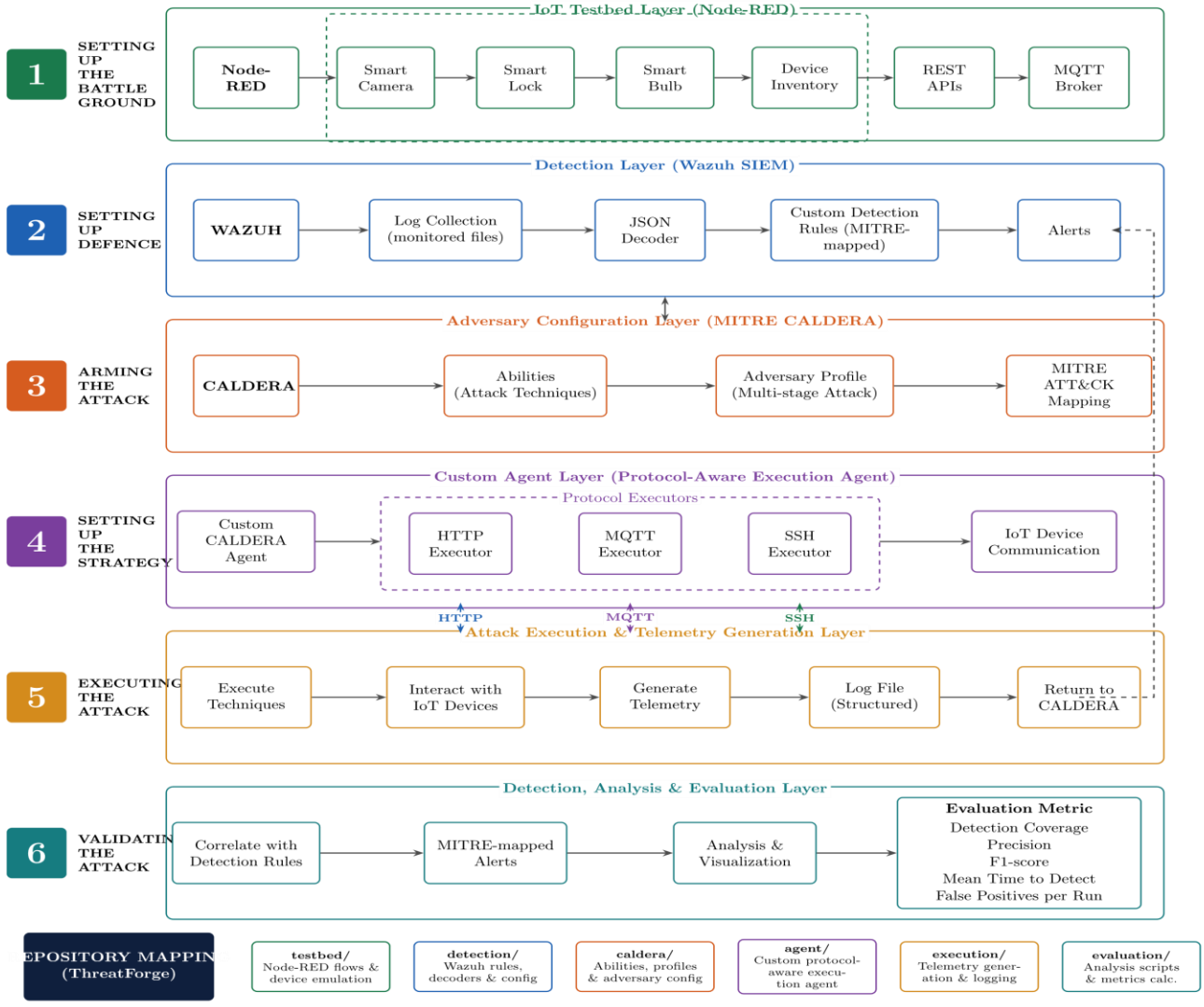


Figure 1. Overall framework architecture.

As illustrated in Fig. 1, Layer 1 establishes the battleground: a Node-RED testbed that simulates a smart-home environment, camera, lock, bulb, and device inventory. Layer 2 constitutes the defence: a Wazuh SIEM pipeline that ingests telemetry and evaluates events against custom, MITRE-mapped detection rules. Layer 3 encodes the offence: a five-stage ATT&CK-mapped adversary profile implemented in MITRE CALDERA. Layer 4 defines the interaction strategy: native command dispatchers for direct IoT communication. Layer 5 drives the attack chain and generates structured telemetry, while Layer 6 closes the loop by correlating detections against ground truth. This layered architecture maps directly onto the components released in the ThreatForge repository.

3.2. Six-Stage Development Pipeline

The methodology is structured around six stages. Fig. 2 outlines this end-to-end pipeline, from agent design through to the correlation structure the completed system supports.

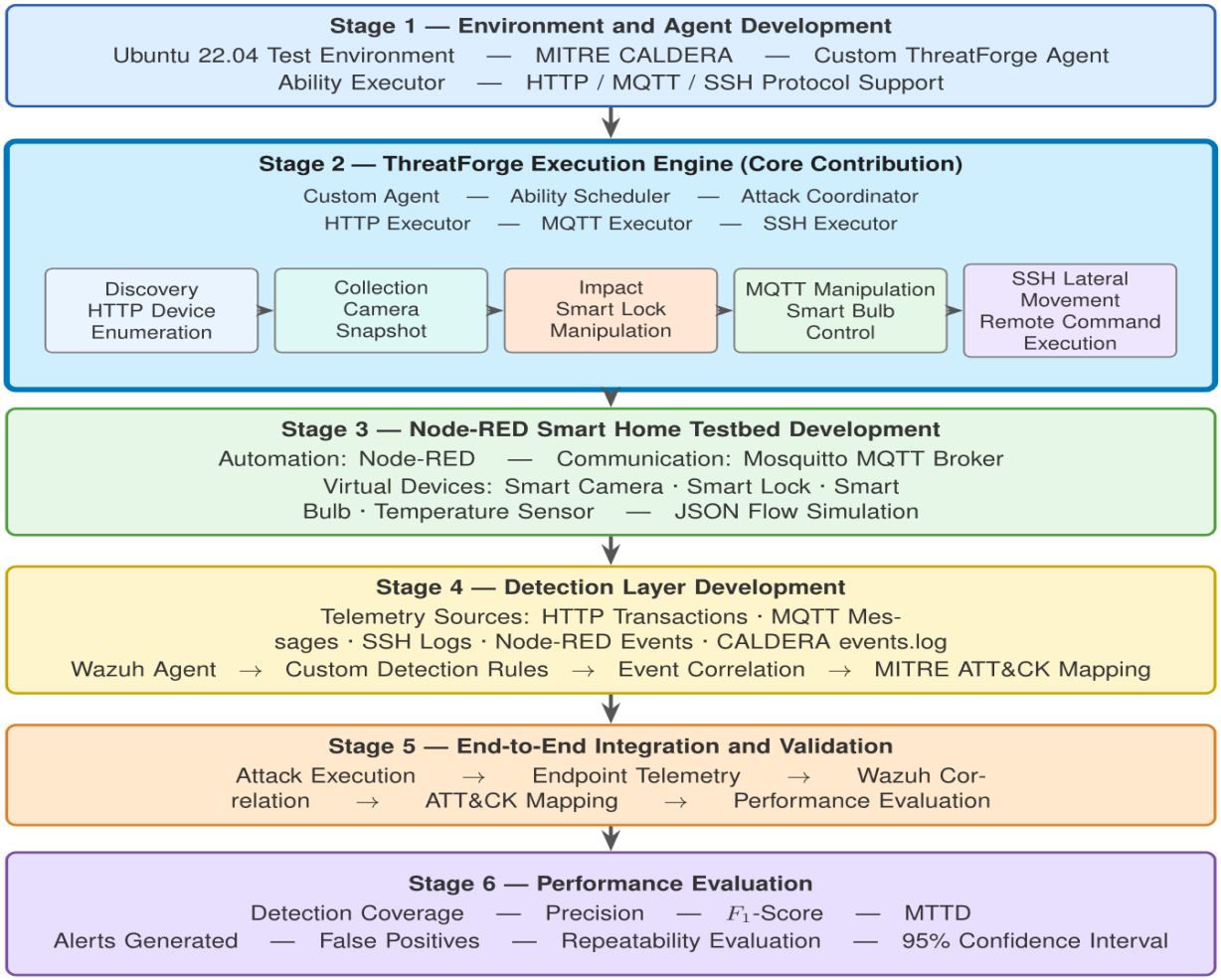


Figure 2. Six-stage development pipeline, from agent design through to correlation structure.

The first stage develops a custom agent comprising five components: an externalised environment configuration layer; a core controller responsible for registration, beaconing, and dispatch orchestration; and three protocol-specific executors covering HTTP, MQTT, and SSH. Each component holds a single responsibility, so that the dispatch logic in the core controller remains untouched by the behaviour of the protocol executors. The second stage designs the ability and adversary profile: five discrete abilities, each mapped to a distinct ATT&CK technique, are composed into a single Custom IoT Adversary profile that progresses as an intrusion — reconnaissance, collection, impact, and lateral movement — so that any operation derived from it produces an ordered event sequence suited to later timeline analysis. The third stage designs the testbed around four endpoint handlers: device reconnaissance, camera snapshot, smart-lock control, and smart-bulb messaging, each following the dual-output design. The fourth stage designs the detection layer, in which telemetry ingestion and rule logic are structured so that each distinct telemetry outcome maps to a MITRE-tagged alert. The fifth stage defines the integration logic that binds these components together, so that every technique in the adversary profile has a traceable path to at least one corresponding detection rule. The sixth stage defines

the correlation structure: the framework is built to generate three logically independent data streams — an adversary operation log, the testbed's own telemetry log, and the SIEM alert log.

The testbed yields a device-level response to the agent and a structured telemetry record. The telemetry can be consumed directly by Wazuh's decoding layer without any intermediate transformation. The custom agent is architected to remain compatible with CALDERA's existing model. Its design separates protocol-handling logic from core beacon logic, so that the set of supported IoT protocols can work without altering the agent's core identity. Every offensive ability in the system maps to one ATT&CK technique, giving each executed action a reference in the adversary profile and detection layer. This architecture keeps the offensive and defensive halves of the system aligned.

3.3. ThreatForge Framework

ThreatForge is the realisation of the architecture specified in Sections 3.1 through 3.3: each architectural layer, pipeline stage, and design requirement described above corresponds directly to a component within the released framework. The framework is organised with dedicated components for the CALDERA agent, the CALDERA abilities and adversary profile, the Node-RED testbed, the Wazuh detection rules and decoders, and the supporting deployment and documentation layer.

Tab. 2 summarises these components in terms of their research purpose. Rather than enumerating every file in the framework, the table is organised around the categories of architectural artefacts needed to understand and reconstruct the reported system design.

Table 2. ThreatForge repository structure and research artefacts.

Phase	Module	Repository Component	Purpose
1	Custom Agent	hikvision-agent.py, agent.env, http-exec.py, mqtt-exec.py, ssh-exec.py	Protocol-aware CALDERA agent (HTTP, MQTT, SSH executors)
2	Adversary Profile	device_recon.yml, camera_snapshot.yml lock_unlock.yml, bulb_control.yml, lateral_movement.yml	Five-stage Custom IoT Adversary ability and profile definitions
3	IoT Testbed	Node-RED flows.json	Simulated smart-home devices: camera, lock, bulb, inventory
4	Detection Layer	json_decoder.xml, local_rules.xml	CALDERA log decoding and MITRE-mapped Wazuh detection rules
5	Integration	events.log, auth.log	Correlation keys, environment configuration, ability-to-rule mapping
6	Deployment	docker/, README.md	Docker deployment configuration and setup documentation

3.4. Conceptual Applicability of the Architecture

Although evaluated in a Node-RED-based simulation for safety, control, and reproducibility, the proposed architecture is not tied to the simulated environment. Its protocol-aware agent, IoT-specific attack abilities, and detection logic are designed around communication and telemetry patterns applicable to comparable smart-home deployments. The architecture therefore provides a basis for extending the same attack-telemetry-detection workflow to IoT devices. Such validation remains necessary to determine how hardware, firmware, vendor implementations, and network conditions influence detection performance. This is particularly relevant to real-world IoT security, where attacks can affect device integrity, availability, and user privacy.

4. Research Implementation

4.1. Custom Agent Implementation

The custom agent acts as a working extension of the CALDERA beacon and tasking contract, adding an internal dispatch layer that routes decoded instructions to protocol-specific executors. At start-up, the agent loads its runtime parameters, server address, agent group, beacon interval, TLS behaviour, and credential placeholders from a single external configuration file, allowing the same codebase to be redeployed across environments through configuration substitution alone. A stable agent identifier is derived from the host identity and cached for the process lifetime, giving the agent a consistent identity across restarts, as shown in Fig. 3.

Beaconing follows CALDERA's standard communication model: the agent's profile is transmitted to the server on each interval, and returned instructions are unpacked and normalised before dispatch. A placeholder-substitution step lets ability definitions reference credentials resolved at runtime rather than hard-coded values. The dispatcher inspects each decoded instruction and routes it to the matching protocol executor, falling back to direct shell execution when no protocol prefix is recognised. The beacon-dispatch loop is wrapped in fault-tolerant handling that logs failures and retries after a fixed backoff rather than terminating, keeping the agent resilient to transient network or device errors.

Protocol-specific behaviour is implemented through three executors: the HTTP executor supports only the GET and POST methods needed for the testbed's REST interface, returning a normalised status-and-body response that Wazuh can evaluate without full transaction logging. The MQTT executor targets a single control-topic pattern with a binary ON/OFF payload vocabulary, performing a stateless connect-publish-disconnect cycle per invocation that favours predictable, low-persistence execution over connection reuse. The SSH executor provides lateral-movement capability; in the absence of an explicit command, it plants a sentinel file on the target host, giving a low-risk indicator of remote code execution for later detection validation.

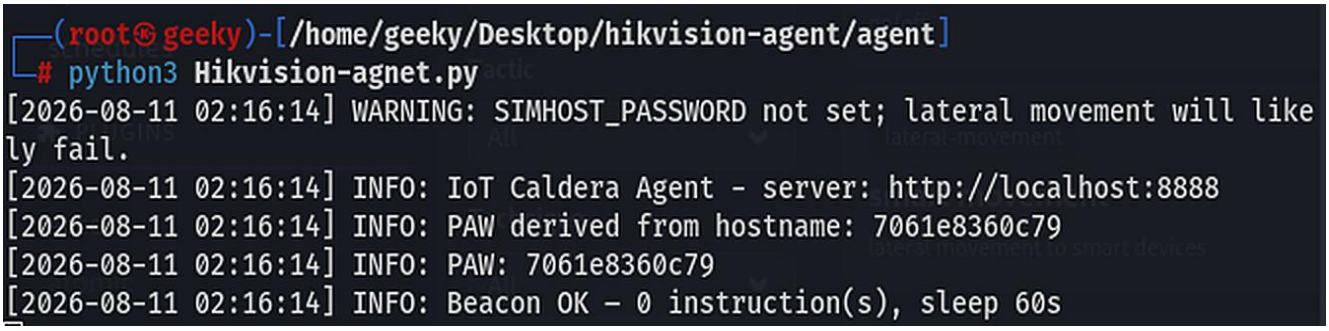


Figure 3. Agent beaconing with the PAW ID.

4.2. Ability and Adversary Profile Configuration

Each ability is implemented as a YAML specification that combines its ATT&CK tactic and technique identifier with a command expressed through the agent's shared dispatcher. A facts-variable placeholder represents the testbed's network address, allowing the same ability definition to run unmodified across deployments.

The five abilities were composed into a single adversary profile, Custom IoT Adversary, following CALDERA's standard model of an adversary as an ordered collection of abilities. Profile creation used the CALDERA web interface. The profile orders execution from reconnaissance through collection and impact, concluding with lateral movement; server-side validation confirmed that all five

abilities registered correctly with their ATT&CK tactic tags intact. Tab. 3 summarises the resulting ability set, authored to exercise every protocol executor and every testbed endpoint at least once; Fig. 4 shows the abilities as configured within CALDERA.

Table 3. Custom ability set and MITRE ATT&CK / ICS mapping.

Ability	Protocol	ATT&CK Tactic	Technique
Device Discovery	HTTP	Reconnaissance	T1595 – Active Scanning
Camera Snapshot Collection	HTTP	Collection	T1125 – Video Capture
Smart Lock PIN Attempt	HTTP	Impact	T0836 – Modify Parameter
Smart Bulb Manipulation	MQTT	Impact	T0831 – Manipulation of Control (ICS)
Lateral Movement (SSH)	SSH	Lateral Movement	T1021.004 – SSH

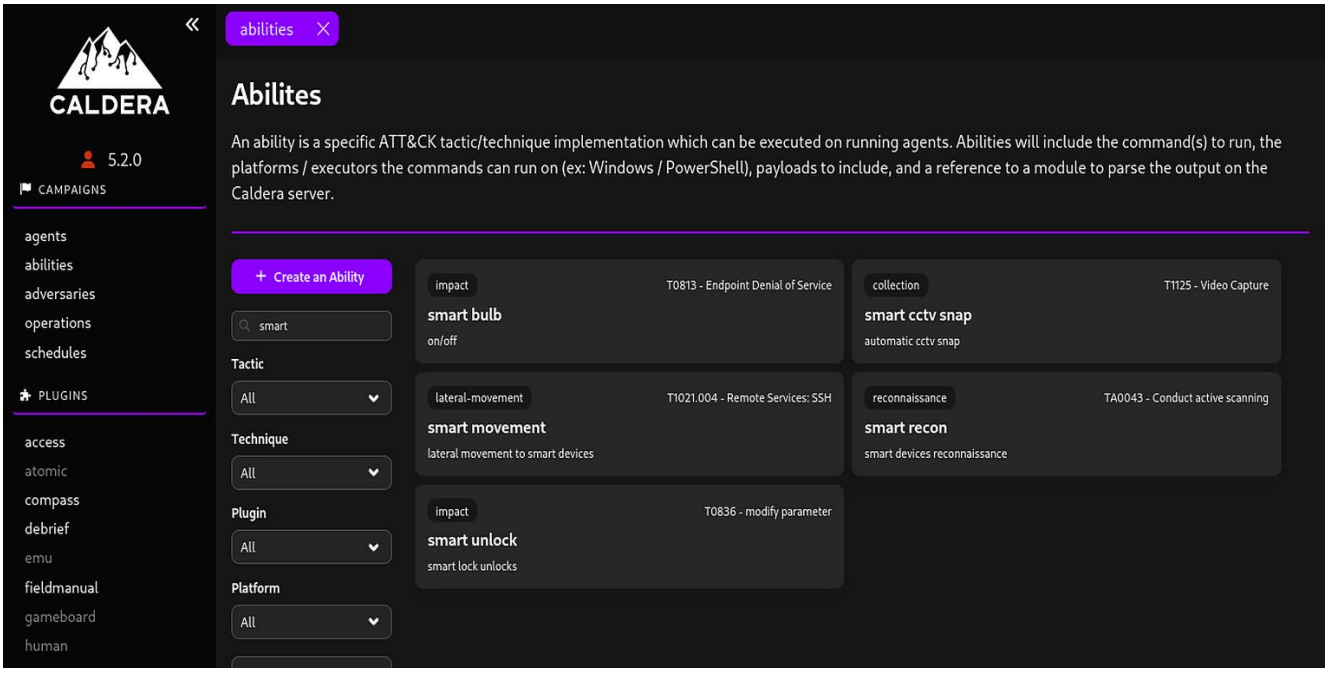


Figure 4. IoT-specific custom abilities created in CALDERA.

4.3. IoT Testbed Implementation

The Node-RED testbed's Layer 1 design reproduces the functional behaviour of a Hikvision-style smart-home gateway without physical hardware, through four endpoint handlers that each follow the dual-output pattern specified in Section 3.2 (Fig. 5).

The Device Recon handler responds to inventory requests with a static device list while emitting telemetry tagged as a reconnaissance event. The Snapshot handler returns a placeholder base64-encoded image on request and emits telemetry flagged with a large-response indicator, supporting exfiltration-oriented detection logic. The Lock handler validates a submitted PIN against a stored value, returns an unlocked or failed response, and logs both the outcome and the attempted PIN, a realistic information-leak pattern that feeds the forensic analysis in Section 6. The Bulb handler subscribes to an MQTT topic and emits impact-tagged telemetry on receipt of a valid ON/OFF payload; because MQTT's fire-and-

forget model has no request-response pairing, this handler produces telemetry without an accompanying HTTP response. Lateral movement is excluded from the Node-RED layer: the SSH executor interacts directly with the companion host's SSH daemon, and its activity is captured through native authentication logs rather than testbed telemetry.

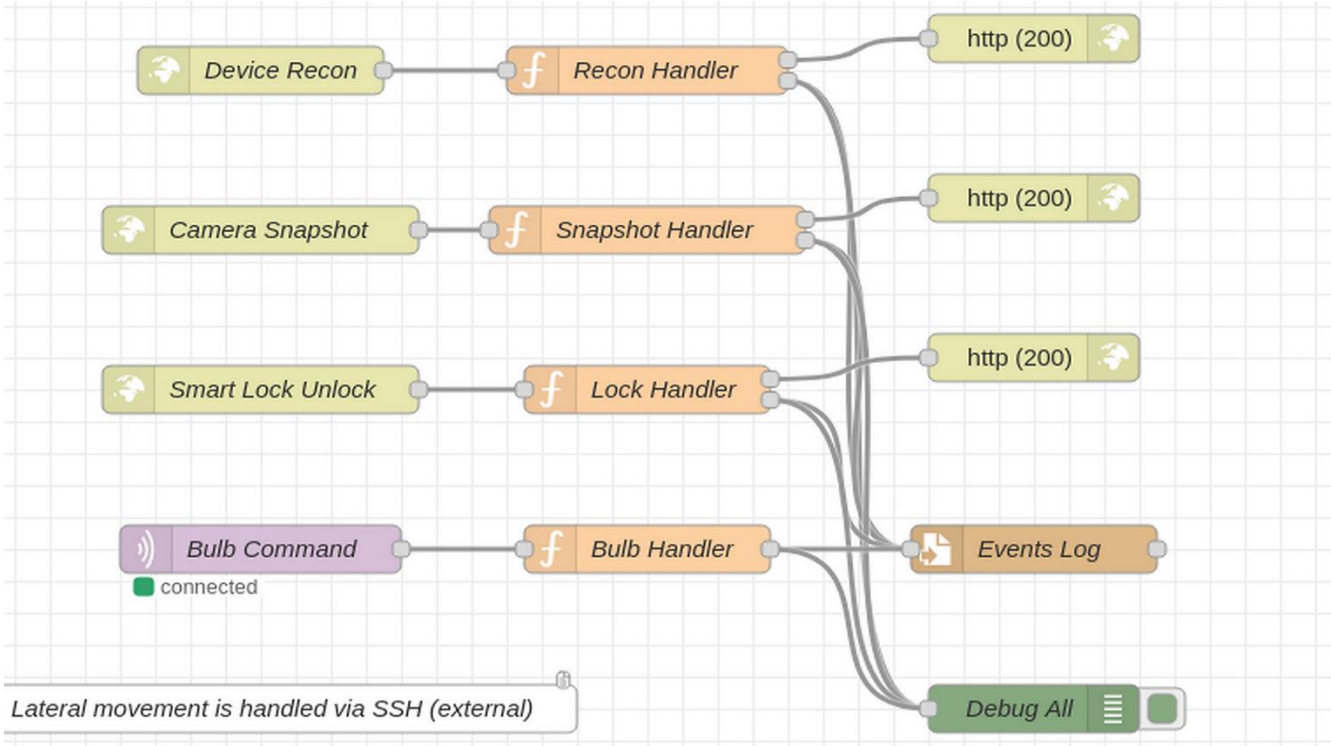


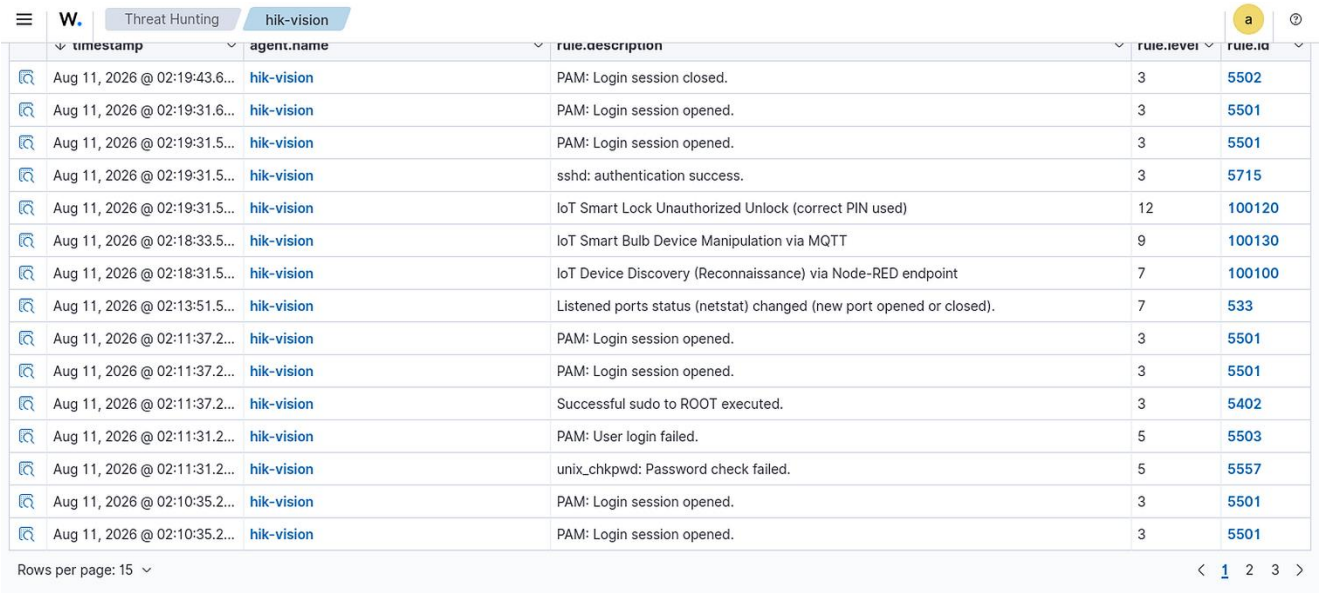
Figure 5. Node-RED IoT testbed.

4.4. Detection Layer Implementation

The detection layer implements Wazuh's standard log collection, decoding, rule evaluation, and alerting pipeline; no custom decoder is introduced, since Wazuh's native JSON decoder already maps each top-level telemetry field into a rule field. A log-collector configuration directs the Wazuh agent to ingest the testbed's telemetry as JSON, and detection logic follows the one-rule-per-outcome principle set out in Section 3.2, summarised in Tab. 4 and shown in operation in Fig. 6.

Table 4. Detection rule design and MITRE mapping.

Ability	Rule ID	MITRE Technique	Rule Condition
Device Discovery	100100	T1595	Inventory endpoint accessed
Camera Snapshot Collection	100110	T1125	Snapshot response exceeds size threshold
Smart Lock PIN Attempt	100120	T0836	Lock endpoint invoked with PIN payload
Smart Bulb Manipulation	100130	T0831	Valid ON/OFF payload on bulb topic
Lateral Movement (SSH)	5715 / 5501	T1021.004 – SSH	Native Wazuh SSH authentication / file-integrity rule



timestamp	agent.name	rule.description	rule.level	rule.id
Aug 11, 2026 @ 02:19:43.6...	hik-vision	PAM: Login session closed.	3	5502
Aug 11, 2026 @ 02:19:31.6...	hik-vision	PAM: Login session opened.	3	5501
Aug 11, 2026 @ 02:19:31.5...	hik-vision	PAM: Login session opened.	3	5501
Aug 11, 2026 @ 02:19:31.5...	hik-vision	sshd: authentication success.	3	5715
Aug 11, 2026 @ 02:19:31.5...	hik-vision	IoT Smart Lock Unauthorized Unlock (correct PIN used)	12	100120
Aug 11, 2026 @ 02:18:33.5...	hik-vision	IoT Smart Bulb Device Manipulation via MQTT	9	100130
Aug 11, 2026 @ 02:18:31.5...	hik-vision	IoT Device Discovery (Reconnaissance) via Node-RED endpoint	7	100100
Aug 11, 2026 @ 02:13:51.5...	hik-vision	Listened ports status (netstat) changed (new port opened or closed).	7	533
Aug 11, 2026 @ 02:11:37.2...	hik-vision	PAM: Login session opened.	3	5501
Aug 11, 2026 @ 02:11:37.2...	hik-vision	PAM: Login session opened.	3	5501
Aug 11, 2026 @ 02:11:37.2...	hik-vision	Successful sudo to ROOT executed.	3	5402
Aug 11, 2026 @ 02:11:31.2...	hik-vision	PAM: User login failed.	5	5503
Aug 11, 2026 @ 02:11:31.2...	hik-vision	unix_chkpwd: Password check failed.	5	5557
Aug 11, 2026 @ 02:10:35.2...	hik-vision	PAM: Login session opened.	3	5501
Aug 11, 2026 @ 02:10:35.2...	hik-vision	PAM: Login session opened.	3	5501

Rows per page: 15 1 2 3

Figure 6. Wazuh SIEM: Hikvision agent activity monitoring.

4.5. Integration Verification and Execution

Prior to generating live attack traffic, a telemetry entry was injected into Wazuh's rule-testing utility to confirm that each rule fired with the correct MITRE tag. This was followed by a check across all five abilities, confirming that each had a complete, unbroken path from its executor, through the corresponding testbed handler or host-level log source, to at least one detection rule and alert (Fig. 7). Carried out ahead of the live evaluation in Section 6, this step ensures that the reported results reflect genuine end-to-end detection performance.

```
[2026-08-11 02:18:30] INFO: Ability 6a4ae032-e68a-470f-9f23-7d487bdb9852: http GET
http://localhost:1880/api/devices
[2026-08-11 02:18:31] INFO: Result for 6a4ae032-e68a-470f-9f23-7d487bdb9852 posted
(exit=0)
[2026-08-11 02:18:31] INFO: Ability a4ce4547-6c53-4dba-a863-2f75bec59a06: http POST
 http://localhost:1880/api/camera/snapshot
[2026-08-11 02:18:32] INFO: Result for a4ce4547-6c53-4dba-a863-2f75bec59a06 posted
(exit=0)
[2026-08-11 02:18:32] INFO: Ability f4abf642-7502-4c7b-89d5-654bbb966759: mqtt loca
lhost OFF
[2026-08-11 02:18:33] INFO: Result for f4abf642-7502-4c7b-89d5-654bbb966759 posted
(exit=0)
[2026-08-11 02:19:30] INFO: Beacon OK — 2 instruction(s), sleep 47s
[2026-08-11 02:19:30] INFO: Ability d5172709-fd9c-46cd-bfb1-cc3674ad07fd: curl -s -
X POST http://localhost:1880/api/lock/unlock -H "Content-Type: application/json" -d
 '{"pin":"1234"}'
[2026-08-11 02:19:30] INFO: Generic shell command: curl -s -X POST http://localhost
:1880/api/lock/unlock -H "Content-Type: application/json" -d '{"pin":"1234"}'
[2026-08-11 02:19:30] INFO: Result for d5172709-fd9c-46cd-bfb1-cc3674ad07fd posted
(exit=0)
[2026-08-11 02:19:30] INFO: Ability cf20d57b-4d18-4baa-b738-cd376440f2af: ssh 127.0
.0.1 simhost kali2024
[2026-08-11 02:19:32] INFO: Result for cf20d57b-4d18-4baa-b738-cd376440f2af posted
(exit=0)
```

Figure 7. Integrity verification using custom CALDERA agent logs.

5. Experimental Setup

5.1. Environment Description

The evaluation is conducted on the deployed ThreatForge system described in Section 4, comprising four subsystems: the Node-RED IoT testbed, the Wazuh SIEM detection layer, the MITRE CALDERA adversary-emulation server, and the custom protocol-aware agent. All components are hosted within an isolated laboratory network, separated from production and external traffic, so that observed telemetry and alerts can be attributed to the adversary operations executed. The testbed, SIEM manager, CALDERA server, and companion host each run as independent components within this environment, communicating over the same protocol interfaces, HTTP, MQTT, and SSH, that the framework is designed to emulate.

5.2. Baseline Conditions

Before executing any adversary operation, a one-hour pre-attack baseline window is established, with no adversary activity present. During this window, the background alert rate, host resource utilisation, and any false-positive detections are recorded, establishing a reference against which alert activity generated during attack execution can later be assessed in Section 6. All monitoring components, the Wazuh log collector, the CALDERA server, and the testbed's telemetry logger, remain active in their normal operating configuration throughout the baseline window, ensuring that the observed conditions reflect the system's steady-state behaviour.

5.3. Attack Execution Conditions

Each experimental trial executes the complete Custom IoT Adversary profile defined in Section 4.2, comprising all five links, reconnaissance, collection, the two impact actions, and lateral movement, in a fixed execution order. Execution is initiated through the CALDERA operations interface and proceeds with the operator choosing to run each ability individually or execute the entire adversary kill chain, so that each trial follows an identical, automated sequence of technique invocations under the same test configuration (Fig. 8). To assess repeatability, this operation is executed across 25 repeated trials in laboratory conditions, with each trial treated as an independent observation of the same underlying attack chain.

5.4. Data Collection Method

Three data streams are collected for every trial: the CALDERA operation log, which records the execution timestamp, status, and output of each adversary action; the Node-RED testbed's own telemetry log, which records each device-level interaction as observed by the testbed; and the Wazuh alert log, which records every security alert generated for the monitored agent across the full observation window. All three components are synchronised to a common system clock ahead of each trial. This correlation structure follows directly from the design specified in Section 3.2 and provides the basis for the metric computation described below.

5.5. Ground-Truth Establishment and Metric Definitions

Three data streams are collected during each trial: (i) CALDERA operation logs, which record the executed ability, execution timestamp, reported status, and unique operation identifier; (ii) Node-RED testbed telemetry, which records device-level interactions, including HTTP requests/responses and MQTT messages, together with the associated correlation key and timestamp; and (iii) Wazuh alert logs,

which record generated alerts, including the correlation key, rule identifier, MITRE ATT&CK mapping, and timestamp.

The collected logs were processed to establish a per-technique record for each experimental trial. CALDERA logs provided the identity of the executed ability and its execution timestamp. The corresponding Node-RED telemetry logs were then examined to verify that the expected device or system activity occurred and to establish its occurrence time. Wazuh logs were examined for alerts associated with the same ATT&CK technique and occurring within the evaluation window. Alert rule identifiers and MITRE ATT&CK mappings were used to distinguish the expected detection from unrelated alerts. This process converted the raw execution, telemetry, and SIEM logs into a structured set of technique-level outcomes.

A **240-second evaluation window** was applied to each complete run. Each five-technique attack sequence typically required approximately 180 seconds, allowing an additional 60 seconds for telemetry propagation, processing, and Wazuh alert generation.

For each technique instance across the 25 trials, **True Positive (TP)** was recorded when the expected activity and corresponding Wazuh alert occurred within the 240-second window. **False Negative (FN)** was recorded when the expected activity occurred but no corresponding alert was generated within the window. **False Positive (FP)** was recorded when a Wazuh alert could not be associated with executed adversary techniques- alerts caused by background or non-adversarial activity.

Framework performance is assessed using five metrics, each computed from the correlated data streams described above. **Detection Coverage (Recall)** captures the proportion of executed adversary techniques that produced at least one corresponding detection alert, reflecting the breadth of the detection layer's visibility into the attack chain. **Detection Precision** captures the proportion of generated alerts that correctly correspond to an executed adversary technique, reflecting the accuracy of the alerting logic under attack conditions. **F1-score** combines coverage and precision into a single measure, capturing the trade-off between detecting the full attack chain and avoiding excessive or mismatched alerting. **Mean Time to Detect (MTTD)** captures the interval between the execution timestamp of an adversary action and the timestamp of its corresponding detection alert, reflecting the responsiveness of the detection pipeline. **False Positives per Run** captures the number of alerts generated that do not correspond to any executed adversary action within a given trial, providing a measure of alerting noise under the experimental conditions established in Section 5.2.

The resulting measurements across the 25 trials are summarised using the mean, standard deviation, and 95% confidence interval for each metric, providing a statistical basis for assessing consistency of framework performance; the corresponding values are reported in Section 6.

Figure 8. Agent executing the five-stage adversary profile.

6. Results

6.1. Baseline System Behaviour

The baseline provides a reference point for assessing subsequent alert activity generated during adversary execution. Over the one-hour pre-attack observation period, the testbed generated an average of 15.2 host and network events per hour, consisting of routine operating-system processes, periodic MQTT communications, and Node-RED middleware activity. Mean host CPU utilisation remained stable at 12.4%, with Wazuh telemetry collected at 60-second intervals.

No custom IoT detection rules were triggered during this period. Although routine operating-system events continued to be logged by Wazuh, none satisfied the conditions defined by the custom rule set. This separation between background activity and rule-triggering events indicates that the custom rules respond specifically to adversary behaviour, not routine operations.

6.2. Representative Adversary Execution

A representative execution of the five-stage adversary profile, device discovery, camera snapshot collection, smart-lock manipulation, MQTT-based smart-bulb control, and SSH-based lateral movement was completed in approximately 180 seconds, providing an end-to-end view of framework behaviour across a full attack chain.

CALDERA's orchestration log reported all five abilities as successfully executed. Endpoint-side verification confirmed this outcome: the Node-RED execution log recorded the events, and Wazuh generated the associated high-severity alert, as shown in Fig. 9, which presents the endpoint summary and corresponding MITRE ATT&CK tactic mapping. Reported orchestration success is not necessarily equivalent to target-side execution, and endpoint telemetry provides a more reliable indication of what actually occurred.

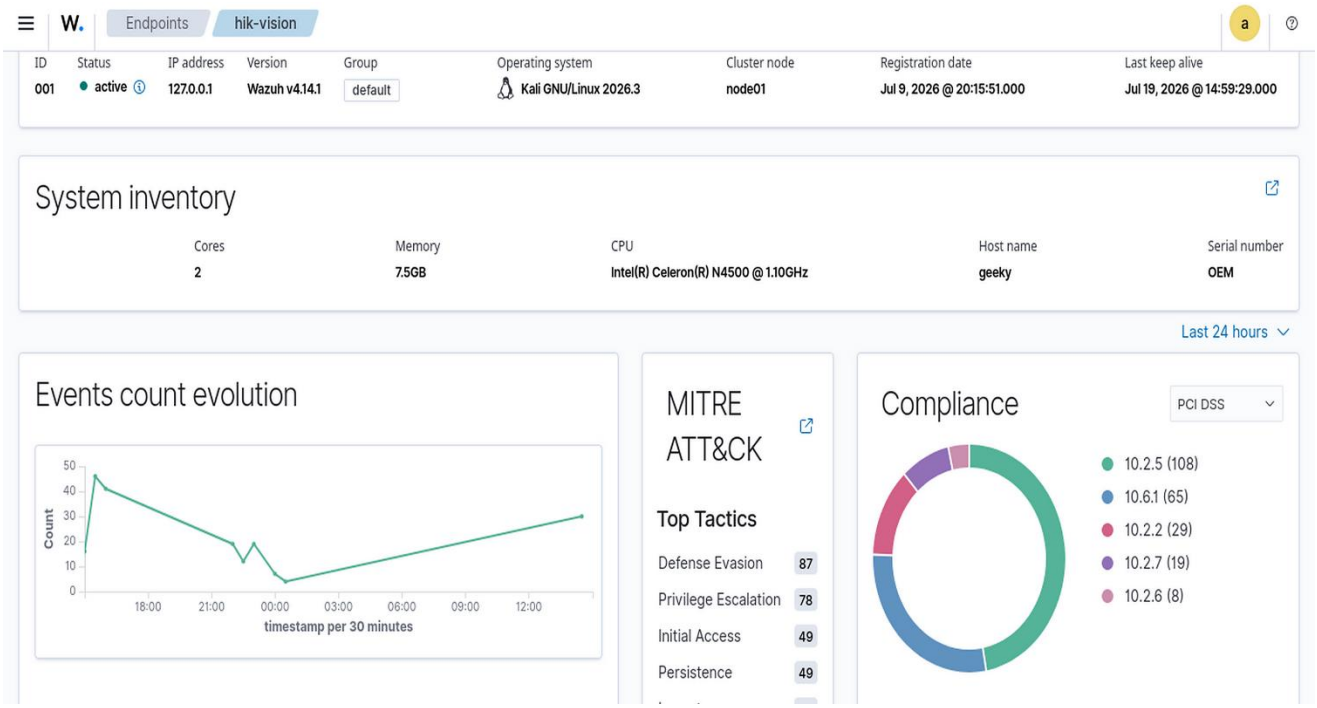


Figure 9. Hikvision agent endpoint summary, system inventory & MITRE ATT&CK tactics.

All five ATT&CK techniques produced corresponding security telemetry within the monitoring window. A total of 194 alerts were recorded across the execution, comprising custom IoT detections, native Wazuh host-security notifications, and routine operating-system events. As illustrated in Fig. 10, Wazuh detected all five evaluated techniques.

Figure 10. Threat hunting summary and alert timeline.

6.3. Multi-Run Statistical Results

While Section 6.2 demonstrates end-to-end detection in a single execution, repeated trials assess whether this behaviour is consistent. The complete adversary profile was executed across 25 runs under identical laboratory conditions, covering 125 total technique instances. Of these, 118 were successfully detected, corresponding to an aggregate detection coverage of 94.4%. The observed missed detections were associated with identifiable telemetry and log-processing conditions, including duplicate MQTT messages, decoder failures, and event-timing effects.

Across the 25 runs, detection performance follows a consistent pattern. Missed detections were not uniformly distributed: some runs captured all five techniques, while others captured fewer, producing the difference between aggregate and per-run coverage. Overall, the narrow confidence intervals across all metrics indicate that ThreatForge's detection behaviour is stable under controlled conditions.

Tab. 5 summarises the statistical performance across all 25 runs; mean detection coverage was 94.4% ± 9.2% (95% CI: 90.6%–98.2%). Mean precision was 89.8% ± 10.4% (95% CI: 85.5%–94.1%), with a corresponding mean F1-score of 91.6% ± 7.5% (95% CI: 88.5%–94.7%). Detection latency remained stable at (59.0 ± 7.0) s (95% CI: 56.1 s–61.9 s). The framework generated an average of 0.60 ± 0.65 false-positive alerts per run (95% CI: 0.34–0.86), indicating low false-positive volume.

These results demonstrate that ThreatForge can execute and detect adversary techniques in an IoT testbed while maintaining low alert noise, consistent with effective integration of adversary emulation and SIEM-based detection. Overall, the findings support ThreatForge as a practical approach for validating IoT security monitoring and detection capabilities under controlled attack scenarios.

Table 5. Statistical performance summary across 25 experimental runs.

Across the 25 runs and 125 trials, True Positives = 118; False Negatives = 7; False Positives = 15

Metric	Interpretation	Formula	Calculation	Mean	SD	95% CI
Detection Coverage	How many executed attacks were detected	$\frac{TP}{TP + FN}$	$\frac{118}{118 + 7}$	94.4%	9.2%	90.6%–98.2%
Precision	How accurately the alerts corresponded to executed techniques	$\frac{TP}{TP + FP}$	$\frac{118}{118 + 15}$	89.8%	10.4%	85.5%–94.1%
F1-score	Balance between Coverage and Precision	$\frac{2 \times (Coverage \times Precision)}{Coverage + Presicion}$	$2 \times \frac{94.4 \times 88.7}{94.4 + 88.7}$	91.6%	7.5%	88.5%–94.7%
Mean Time to Detect	Time from adversary execution to detection alert	$\frac{Sum\ of\ all\ Detection\ times}{25}$	$\frac{1474.5}{25}$	59.0 s	7.0 s	56.1 s–61.9 s
False Positives per run	Average noise or false alerts	$\frac{Sum\ of\ all\ False\ Positive\ Counts}{25}$	$\frac{15}{25}$	0.60	0.65	0.34–0.86

Metrics were calculated independently for each of the 25 runs and summarised using the mean, sample SD, and two-sided 95% CI (df = 24). Aggregate TP, FN, and FP counts are reported separately; therefore, pooled precision may differ from the reported mean per-run precision.

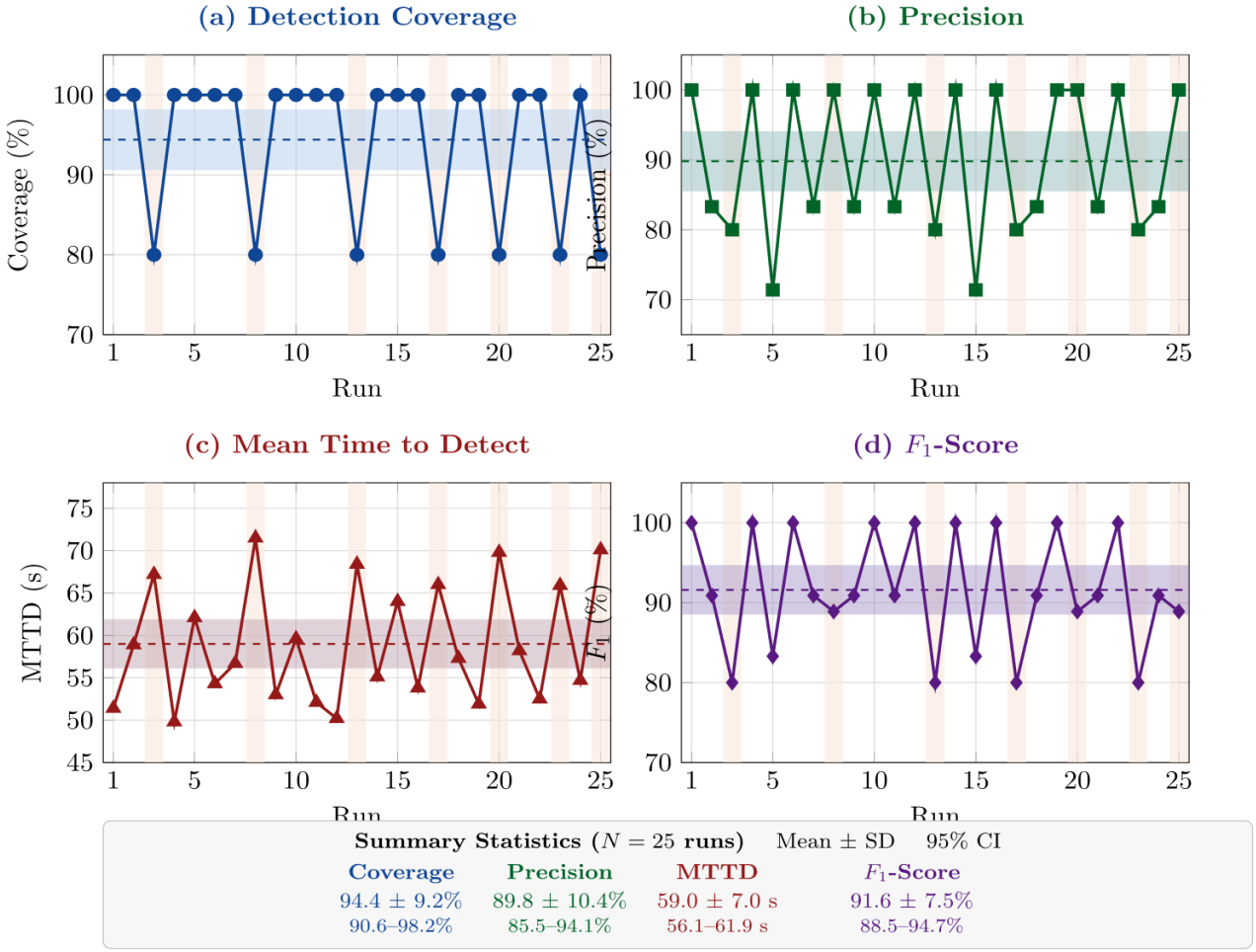


Figure 11. Detection Performance Summary.

7. Discussion

The experimental evaluation demonstrates that the proposed methodology provides a strong basis for assessing IoT detection performance through ATT&CK-aligned adversary emulation. Across 25 repeated runs, detection coverage, precision, and F1-score remained high, while detection latency showed limited variation (Tab. 5). The experimental results are discussed below in relation to the three research questions, focusing on attack reproduction, detection validation, and system responsiveness.

7.1. RQ1: Reproducing ATT&CK-Aligned Behaviour in IoT Environments

Protocol-aware execution enables realistic reproduction of ATT&CK techniques within a representative IoT setting by interacting with devices through native communication protocols. This contrasts with conventional adversary emulation platforms, which are primarily designed for enterprise endpoints and provide limited support for IoT-specific interaction patterns.

The orchestration-layer status alone is insufficient to establish ground truth in IoT environments; reliable validation instead requires correlating orchestration output with endpoint telemetry and the detection layer when evaluating adversary behaviour in IoT contexts.

7.2. RQ2: Validating IoT Detection Capabilities Through Emulation

Correlating ATT&CK-mapped actions with both endpoint telemetry and SIEM alerts enables measurement of detection coverage, precision, latency, and false-positive behaviour across repeated trials. As shown in Section 6.3, missed detections were associated with telemetry arriving outside the evaluation window rather than with failed execution, while false positives were limited and attributable to duplicate MQTT message delivery and decoder-level pattern matching (Fig. 10).

These findings highlight that detection effectiveness in IoT environments depends on the full monitoring pipeline, including telemetry generation, transport, processing, and correlation, rather than on detection logic alone. Adversary emulation therefore functions not only as a validation mechanism but also as a diagnostic tool capable of exposing bottlenecks in the detection pipeline that would remain if evaluation relied on alert counts or execution status.

7.3. RQ3: System Responsiveness and Detection Latency in IoT Environments

Detection latency remained stable across both the representative run and the full set of 25 trials (Tab. 5), indicating consistent system responsiveness under repeated execution. The variation observed across techniques follows a clear pattern: reconnaissance-stage actions were detected rapidly, while SSH-based lateral movement exhibited the highest latency (Fig. 11).

This reflects differences in telemetry pathways. HTTP- and MQTT-based interactions produce near-real-time testbed telemetry, whereas SSH-based activity is detected only after propagation through host-level logging mechanisms, so the resulting latency differences arise from the structure of the monitoring pipeline itself. From an operational perspective, this suggests that improving detection responsiveness in IoT environments may require optimisation of telemetry collection and processing stages.

7.4. Positioning Relative to Existing Frameworks

Tab. 6 compares ThreatForge with established adversary-emulation platforms, including Atomic Red Team, MITRE CALDERA, Prelude Operator, and Infection Monkey. These platforms provide different approaches to ATT&CK-aligned testing and adversary emulation. Atomic Red Team provides

focused ATT&CK-mapped tests that can be executed individually or through an execution framework, while CALDERA provides automated adversary emulation through ATT&CK-aligned abilities and operations [5], [21]. Infection Monkey similarly provides open-source adversary emulation and empirical validation of security controls [22]. The comparison therefore focuses specifically on the capabilities relevant to the present study: IoT protocol-aware execution, integrated detection validation, quantitative evaluation, and reproducibility.

The comparison should not be interpreted as evidence that the existing platforms cannot be extended to support these capabilities. Rather, the distinction is whether such capabilities are explicitly integrated and evaluated as part of the platform or study considered here. ThreatForge's contribution is consequently the integration of protocol-aware IoT execution, ATT&CK-aligned adversary emulation, SIEM-based detection validation, and evaluation within a reproducible experimental workflow.

Table 6. Comparison of adversary-emulation platforms and the proposed framework.
Yes: Supports natively. Partial: Supports with limitations or additional integration. No: Not supported

Evaluation Criterion	Atomic Red Team [21]	MITRE CALDERA [5]	Prelude Operator [6]	Infection Monkey [22]	ThreatForge (Proposed)
ATT&CK-based emulation	Yes	Yes	Yes	Partial	Yes
Native IoT protocol support	No	No	No	No	Yes
Integrated detection validation	No	No	No	No	Yes
Quantitative performance metrics	No	No	No	No	Yes
Reproducible, open-source release	Yes	Yes	Partial	Yes	Yes

The results show that effective IoT security evaluation requires validating not only whether adversary techniques execute, but also how the resulting activity is captured, processed, and interpreted by monitoring systems. By linking adversary execution, telemetry generation, and detection outcomes within a single framework, the proposed methodology provides a reproducible basis for assessing detection performance and identifying pipeline-level limitations.

These findings support the use of closed-loop adversary emulation as a practical approach for evaluating IoT detection engineering, and a foundation for extending this work to larger deployments, additional protocols, and more diverse device ecosystems.

8. Threats to Validity

8.1. Internal Validity

Attack execution records, endpoint telemetry, and SIEM alerts were correlated for each of the 25 runs, so that detections could be checked against ground truth. A related concern is that the same research team authored both the adversary abilities and the detection rules, which risks the two being tuned to each other rather than independently validated. The cross-source correlation used here narrows that risk, and a natural next step would be to test the framework against detection rules written by a separate team.

8.2. Construct Validity

Each metric — coverage, precision, F1-score, MTTD, and false-positive per run was calculated from three separate sources (the adversary log, testbed telemetry, and SIEM alerts) rather than from any one of them alone, so that an incomplete record in one place would not silently distort a metric. The evaluation window itself is a limitation: several of the missed detections discussed in Section 6.3 turned out to be actions that succeeded but whose telemetry simply arrived after the window closed.

8.3. External Validity

The evaluation uses a Node-RED-based smart-home simulation to provide a controlled and isolated environment for ethical security experimentation. This avoids exposing real consumer IoT devices to attack activity while allowing device states, network interactions, and telemetry to be controlled and reproduced consistently across trials. Node-RED has also been used in prior research to simulate and evaluate smart-home IoT vulnerabilities in controlled virtual environments. Broadening the technique set, testing on physical hardware and trying an alternative SIEM would chip away at this gap.

8.4. Conclusion Validity

Twenty-five runs are sufficient to establish a stable pattern in coverage, precision, and latency, but the false-positive per run (0.60 ± 0.65) reflects the inherent variability of counting rare events over a limited number of trials. The false-positive per run should be treated as a rough estimate until it has been assessed over more trials. None of these changes what the results show within the controlled setting they were collected in; it simply delineates the boundaries of that setting. The next step, discussed further in Section 11, is testing the same framework against more realistic conditions: a wider mix of devices, less predictable attack behaviour, and network conditions beyond the laboratory setting.

9. Reproducibility and Availability

To replicate the experiments, the same six-phase workflow described in this paper should be followed. The ThreatForge repository includes the custom agent implementation, adversary definitions, Node-RED flows, Wazuh detection rules, Docker configurations and setup documentation. Reproducing the workflow involves cloning the repository, deploying the environment, importing the provided configurations, and executing the defined attack sequence while capturing the corresponding logs. To support reproducibility, all source code and configuration files are made available through the following repository, ***ThreatForge GitHub repository [23]: [link removed for double-blind review]***

10. Conclusion

This paper introduced ThreatForge, a protocol-aware adversary-emulation framework for evaluating IoT security through ATT&CK-aligned execution and SIEM-based detection validation. The framework extends MITRE CALDERA with native IoT protocol executors, integrates these executors with a reproducible Node-RED smart-home testbed, and connects to a Wazuh detection layer within a single closed-loop pipeline.

In doing so, it addressed a key gap identified in Section 2.5: existing adversary-emulation platforms lack native support for IoT-specific attack execution. ThreatForge combines offensive and defensive capabilities within a unified experimental workflow with a mean detection coverage of 94.4% ± 9.2%, a precision of 89.8% ± 10.4%, an F1-score of 91.6% ± 7.5%, a mean detection latency of (59.0 ± 7.0) s, and a low false-positive rate of 0.60 ± 0.65 alerts per run.

ThreatForge is positioned not as a replacement for platforms such as CALDERA or Atomic Red Team, but as an IoT-focused extension that integrates protocol-aware execution with detection validation in a reproducible form (Section 7.4). The current evaluation therefore establishes the validity of the proposed methodology under controlled conditions rather than providing a comprehensive assessment across the full IoT landscape.

11. Future Work

The limitations identified in Section 8 point to several extensions of the present work. A primary direction is expanding device and protocol coverage. The current evaluation considers three device types, camera, lock, and bulb, over HTTP and MQTT. Extending this to Zigbee-based devices, thermostats, and doorbells would test the protocol-executor design (Section 4.1) against more diverse communication patterns. Moving from simulated devices to commercial IoT ecosystems would further assess whether the framework remains robust under real-world constraints.

A second direction is extending the adversary model with adaptive execution. The current five-technique chain follows a fixed sequence; introducing conditional behaviour would better reflect real intrusion dynamics and allow evaluation of detection strategies under less predictable attack paths. This would also enable comparison of rule-based hybrid detection approaches within a common framework.

A third direction is moving beyond application-layer interaction to firmware-level and physical-layer attack scenarios, which would broaden the range of IoT security scenarios that can be evaluated. These directions position ThreatForge as a broader research platform for IoT security evaluation, supporting future work on device diversity, attack realism, and detection engineering across IoT smart home environments.

Declarations

Acknowledgements
No Acknowledgements.
Funding
This research received no external funding.
Competing Interests
No potential competing interest was reported by the authors.
Author Contributions
The author initiated, designed the study, developed the ThreatForge framework and experimental testbed, conducted the experiments, analysed and interpreted the results, and wrote and revised the manuscript. The author approves the final version and takes full responsibility for the work.
Code Availability
The complete implementation described in this paper is openly available through the ThreatForge repository at [link removed for double-blind review], released under the MIT License.
Data Availability
The experimental logs generated for this study, result summaries, and additional raw log data are available at ThreatForge: [link removed for double-blind review]
AI-assisted technologies:
AI tools were used during manuscript preparation for language quality control, reference management, and plagiarism checking. Scribbr and Grammarly were used for language editing, proofreading and grammar. Zotero and Mendeley were used for reference management. All suggestions and results were manually reviewed and verified. No tool was used to generate research data, experimental results, statistical analyses, or scientific evidence.

References

[1] B. Hammi, S. Zeadally, R. Khatoun, and J. Nebhen, "Survey on Smart Homes: Vulnerabilities, Risks, and Countermeasures," *Computers & Security*, vol. 117, Art. no. 102677, 2022, doi: 10.1016/j.cose.2022.102677.
[2] D. P. Miller, R. Alford, A. Applebaum, H. Foster, C. Little, and B. E. Strom, *Automated Adversary Emulation: A Case for Planning and Acting with Unknowns*. MITRE Corporation, McLean, VA, USA, Jun. 2018. [Online]. Available: https://www.mitre.org/news-insights/publication/automated-adversary-emulation-case-planning-and-acting-unknowns
[3] Y. Jia, L. Xing, Y. Mao, D. Zhao, X. Wang, et al., "Burglars' IoT Paradise: Understanding and Mitigating Security Risks of General Messaging Protocols on IoT Clouds," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2020, pp. 465-481, doi: 10.1109/SP40000.2020.00051.
[4] A. M. Winkler and P. Sharma, "Proactive Threat Detection in Enterprise Systems Using Wazuh: A MITRE ATT&CK Evaluation," *Computers & Security*, vol. 159, Art. no. 104702, 2025, doi: 10.1016/j.cose.2025.104702.
[5] R. Alford, D. Lawrence, and M. Kouremetis, "CALDERA: A Red-Blue Cyber Operations Automation Platform," in *Proc. International Conference on Automated Planning and Scheduling (ICAPS) Demonstration Track*, 2022. [Online]. Available: https://caldera.mitre.org/
[6] M. Landauer, K. Mayer, F. Skopik, M. Wurzenberger, and M. Kern, "Red Team Redemption: A Structured Comparison of Open-Source Tools for Adversary Emulation," in *Proc. IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, 2024. [Online]. Available: https://arxiv.org/abs/2408.15645
[7] A. B. Ajmal, M. A. Shah, C. Maple, M. N. Asghar, and S. U. Islam, "Offensive Security: Towards Proactive Threat Hunting via Adversary Emulation," *IEEE Access*, vol. 9, pp. 126023-126033, 2021, doi: 10.1109/ACCESS.2021.3104260.
[8] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, *MITRE ATT&CK(R): Design and Philosophy*. MITRE Corporation, McLean, VA, USA, Mar. 2020. [Online]. Available: https://attack.mitre.org/docs/ATTACK_Design_and_Philosophy_March_2020.pdf
[9] V. Orbinato, M. C. Feliciano, D. Cotroneo, and R. Natella, "Laccolith: Hypervisor-Based Adversary Emulation With Anti-Detection," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 6, pp. 5374-5387, Nov./Dec. 2024, doi: 10.1109/TDSC.2024.3376129.
[10] R. M. Portase, A. Colesa, and G. Sebestyen, "SpecRep: Adversary Emulation Based on Attack Objective Specification in Heterogeneous Infrastructures," *Sensors*, vol. 24, no. 17, Art. no. 5601, 2024, doi: 10.3390/s24175601.
[11] A. Sivanathan, H. H. Gharakheili, C. Wijenayake, A. Radford, and V. Sivaraman, "Classifying IoT Devices in Smart Environments Using Network Traffic Characteristics," *IEEE Transactions on Mobile Computing*, vol. 18, no. 8, pp. 1745-1759, Aug. 2019, doi: 10.1109/TMC.2018.2866249.
[12] Q. Wang et al., "MPInspector: A Systematic and Automatic Approach for Evaluating the Security of IoT Messaging Protocols," arXiv:2208.08751, 2022. [Online]. Available: https://arxiv.org/abs/2208.08751
[13] D. Y. Huang, N. Apthorpe, G. Acar, F. Li, and N. Feamster, "IoT Inspector: Crowdsourcing Labeled Network Traffic from Smart Home Devices at Scale," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 4, no. 2, 2020, doi: 10.1145/3397333.
[14] F. Callegati et al., "Investigating Operational Technology Attacks as Code," *Empirical Software Engineering*, vol. 30, Art. no. 158, 2025, doi: 10.1007/s10664-025-10713-2.
[15] J. Rahmani, K. O. Detken, and A. Sikora, "An Integrated Testbed for MITRE-Mapped Attack Emulation in Industrial Control Networks," *Sensors*, vol. 26, no. 11, Art. no. 3514, 2026, doi: 10.3390/s26113514.
[16] N. Chouliaras, G. Kittes, I. Kantzavelou, L. Maglaras, G. Pantziou, and M. A. Ferrag, "Cyber Ranges and TestBeds for Education, Training, and Research," *Applied Sciences*, vol. 11, no. 4, Art. no. 1809, 2021, doi: 10.3390/app11041809.
[17] MITRE, "MITRE ATT&CK for ICS," [Online]. Available: https://attack.mitre.org/matrices/ics/
[18] M. El-Hajj, "Leveraging Digital Twins and Intrusion Detection Systems for Enhanced Security in IoT-Based Smart City Infrastructures," *Electronics*, vol. 13, no. 19, Art. no. 3941, 2024, doi: 10.3390/electronics13193941.
[19] MITRE, "CALDERA for OT," 2023. [Online]. Available: https://github.com/mitre/caldera-ot
[20] M. Huang, H. Lee, A. Kundu, X. Chen, A. Mudgerikar, N. Li, and E. Bertino, "ARIoTEDef: Adversarially Robust IoT Early Defense System Based on Self-Evolution Against Multi-Step Attacks," *ACM Transactions on Internet of Things*, vol. 5, no. 3, Art. no. 15, Aug. 2024, doi: 10.1145/3660646.
[21] Red Canary, "Atomic Red Team," GitHub repository and documentation, [Online]. Available: https://github.com/redcanaryco/atomic-red-team
[22] Guardicore, "Infection Monkey: An Open-Source Adversary Emulation Platform," GitHub repository, [Online]. Available: https://github.com/guardicore/monkey
[23] Author, "ThreatForge: An IoT Adversary Emulation Framework for Smart Home Cybersecurity Research," GitHub repository, 2026. [Online]. Available: [link removed for double-blind review]